

# Code-Layout, Readability and Reusability

|               |                                 |
|---------------|---------------------------------|
| Date          | 6 July 2026                     |
| Team ID       | XXXXXX                          |
| Project Name  | Credit Card Approval Prediction |
| Maximum Marks | 5 Marks                         |

## Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

## Code Layout Checklist:

| S.No | Code Quality Parameter    | Description                                   | Followed (Yes / No / Partial) | Remarks                                                          |
|------|---------------------------|-----------------------------------------------|-------------------------------|------------------------------------------------------------------|
| 1    | Consistent Indentation    | Uniform spacing/tabs used throughout the code | Yes                           | Follows PEP 8 standards with strict 4-space indentations.        |
| 2    | Proper File Structure     | Files and folders are logically organized     | Yes                           | Directories separate templates, assets, APIs, and model files.   |
| 3    | Meaningful Variable Names | Variables reflect their purpose clearly       | Yes                           | Variables are named after attributes (e.g. income, dti).         |
| 4    | Function / Method Names   | Functions are descriptively named             | Yes                           | Functions have clear names (e.g. preprocess_input).              |
| 5    | Code Comments             | Inline and block comments explain logic       | Yes                           | Code routes and compliance checks are fully documented.          |
| 6    | Modular Design            | Code is split into reusable functions/modules | Yes                           | Inference models are decoupled from server routing scripts.      |
| 7    | No Redundant Code         | Duplicate or unused code is removed           | Yes                           | Common tasks are centralized in reusable helper methods.         |
| 8    | Error Handling            | Exceptions and errors are handled gracefully  | Yes                           | Try-except blocks capture faults and return clean JSON payloads. |

## Reusable Components / Modules:

| S.No | Component / Module Name | Language / Technology | Where Reused                                                                       | Reusability Level (High / Medium / Low) |
|------|-------------------------|-----------------------|------------------------------------------------------------------------------------|-----------------------------------------|
| 1    | preprocess_input()      | Python / Pandas       | Shared in single prediction API (/predict) and batch auditing API (/predict_batch) | High                                    |

| S.No | Component / Module Name | Language / Technology       | Where Reused                                                                     | Reusability Level (High / Medium / Low) |
|------|-------------------------|-----------------------------|----------------------------------------------------------------------------------|-----------------------------------------|
| 2    | delinquency_override()  | Python / Custom Rule Engine | Integrates with compliance portal batch checking and individual risk evaluations | High                                    |
| 3    | load_resources()        | Python / Pickle File system | Loads scalers and model estimators dynamically at Flask application startup      | High                                    |
| 4    | render_charts()         | JavaScript / Chart.js       | Used to render metrics charts and dial probability metrics on client view tabs   | High                                    |
| 5    | fetch_bureau_records()  | Python / Mock API Manager   | Simulates querying credit history registers from external databases              | Medium                                  |

**Overall Code Quality Assessment:**

| Aspect                   | Rating (1-5) | Comments                                                                    |
|--------------------------|--------------|-----------------------------------------------------------------------------|
| Code Layout & Structure  | 5            | Clean separation of views, templates, models, and controllers.              |
| Readability              | 5            | PEP 8 naming standards, consistent indentation, and clear function headers. |
| Reusability              | 5            | Decoupled preprocessing routines are successfully shared across endpoints.  |
| Documentation / Comments | 5            | Thorough docstrings and comments clarify complex compliance rules.          |
| Overall Score            | 5            | Outstanding quality. Code is production-ready, modular, and maintainable.   |